

Project Report

CSE-1204: Digital Logic Design Lab

Session: 2023-2024

Submitted to:

1. Dr. Suraiya Pervin, PhD, Professor

2. Dr. Ismat Rahman, Associate Professor

Department of Computer Science and Engineering

University of Dhaka

Lab Group: B-10

Submitted By:

Md. Samiul Islam Siam (Roll: 02, Reg No. 2023915945)

Md. Sakhawat Hosen (Roll: 18, Reg No. 2023115961)

Faiaz Ibne Iqbal (Roll: 44, Reg No. 2023315987)

Department of Computer Science and Engineering

University of Dhaka

Date of submission: 21/07/2025

Snake Game Simulation

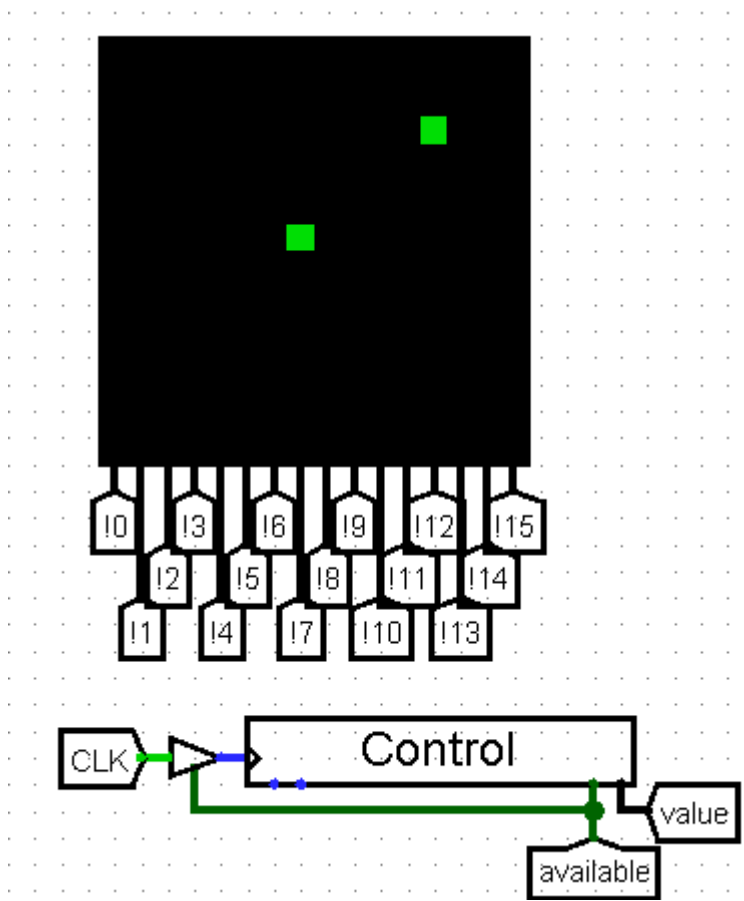
Overview:

This project presents a simple hardware-based Snake game built using **Logisim**, utilizing digital logic components to simulate classic gameplay. A 16×16 LED matrix visually represents the game grid, where a snake moves in response to user inputs from a keyboard interface. Directional control is achieved through logic circuits that interpret key presses (W, A, S, D) and update the snake's position in real time. Registers and memory elements store the snake's coordinates, while combinational logic manages movement, display updates, and basic collision checks.

Apparatus:

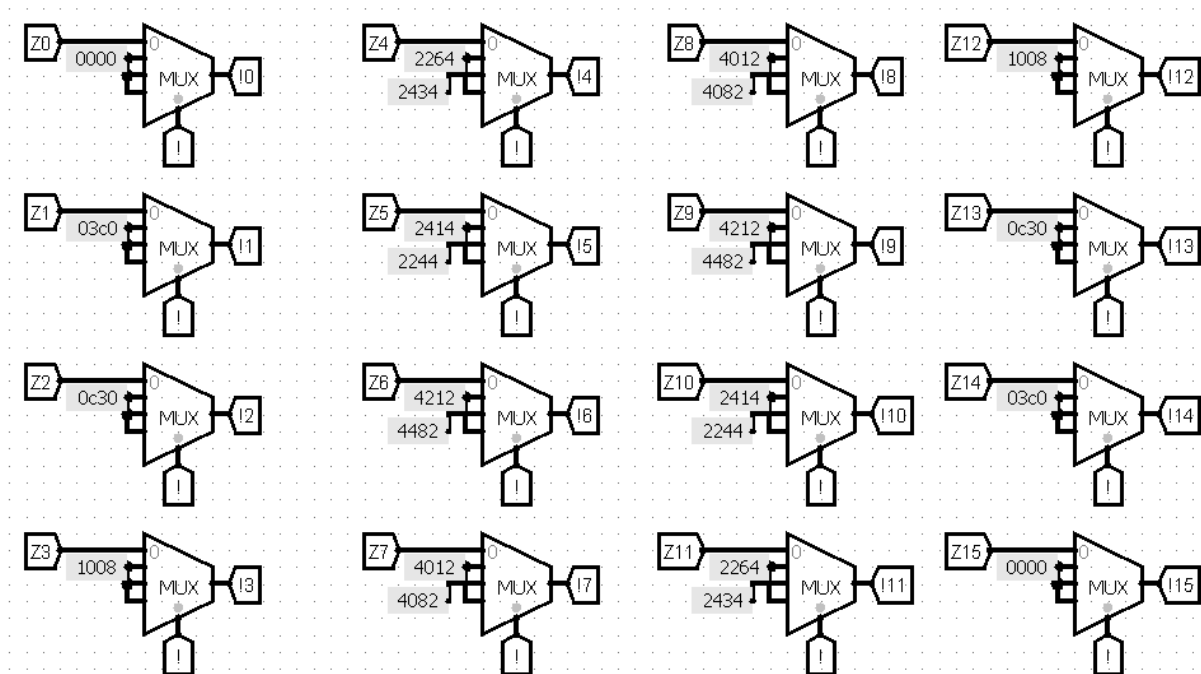
1. 16 x 16 LED Matrix
2. Tunnels
3. Clock Pulse
4. Keyboard Control
5. Random Number Generator
6. Comparator
7. Multiplexer
8. Demultiplexer
9. Basic Logic Gates
10. Splitter
11. Bit Extender
12. Register
13. Adder
14. Subtractor
15. Constants

Circuit Diagrams and their Working Principles



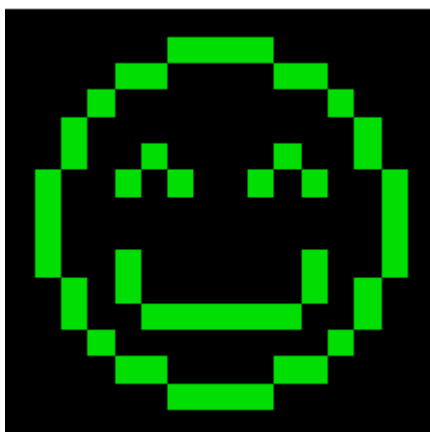
Main Display:

This is the main interface of the project. The top screen is a **16 x 16 LED matrix** where each column is connected to each of the 16 tunnels below. The control takes user input (*W, A, S, D*) and changes the direction of the screen in the next clock pulse. The game runs on **8 Hz tick frequency** for better snake movement. Each time the snake collects the apple, the size increases and the snake become bigger until the snake eats 5 *apples*. Then the player wins and the display shows a smiley face. If the snake touches itself- the game ends and it shows a sad face to indicate loss condition.

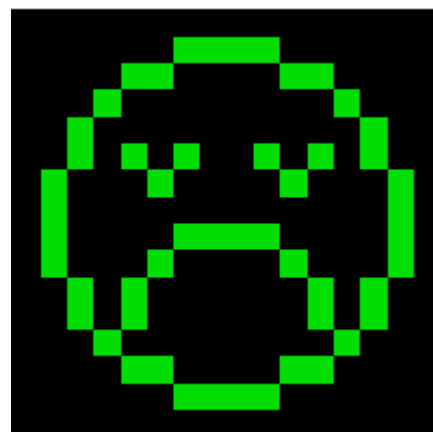


This module updates the tunnels **!(0-15)** to display it on the LED matrix using multiplexers. **Z(0-15)** shows the snake movement and game progression. Other constant values are in the multiplexers to showcase game win and loss screens.

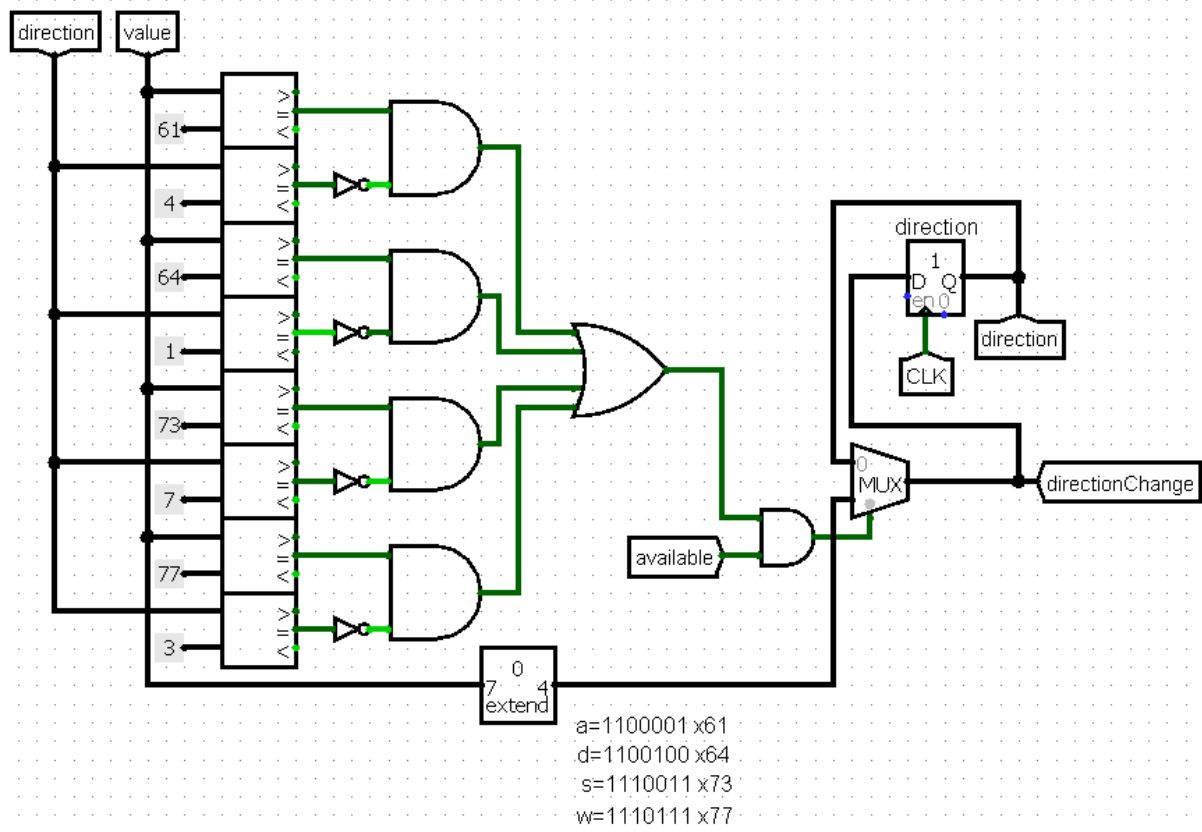
! tunnel is a select bit that modifies the info. The 16 data bit constant values indicate position of the columns to turn on.



If the player wins

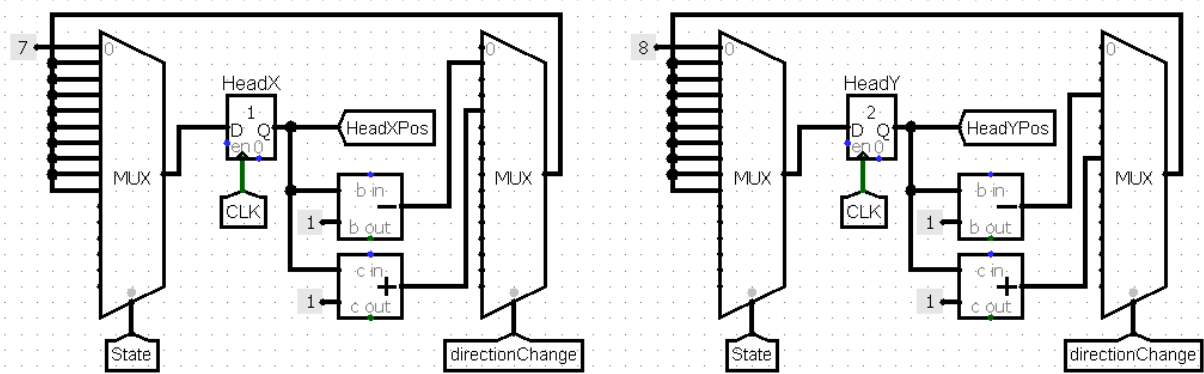


If the player loses



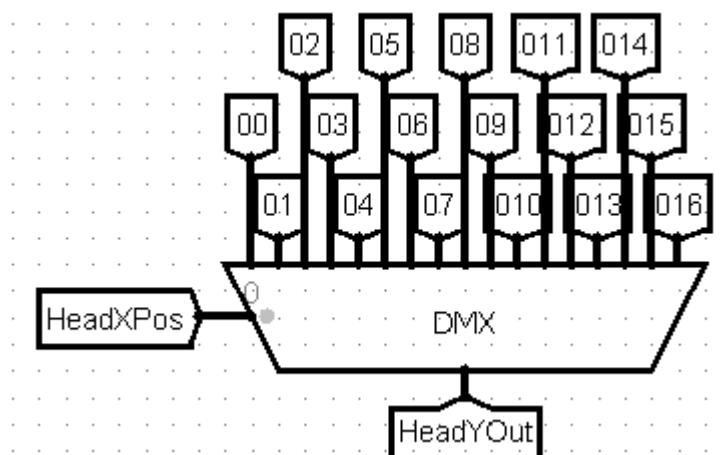
Direction Controller Module:

This module checks if the snake is not going in that direction and then uses the ASCII values of w, a, s, d to change the direction of the snake. Using **bit extender**, the ASCII values are truncated for simplicity. If there is no user input, the snake moves in the same direction as in the register. It takes the user input every clock pulse (using the available tunnel) for proper input management.



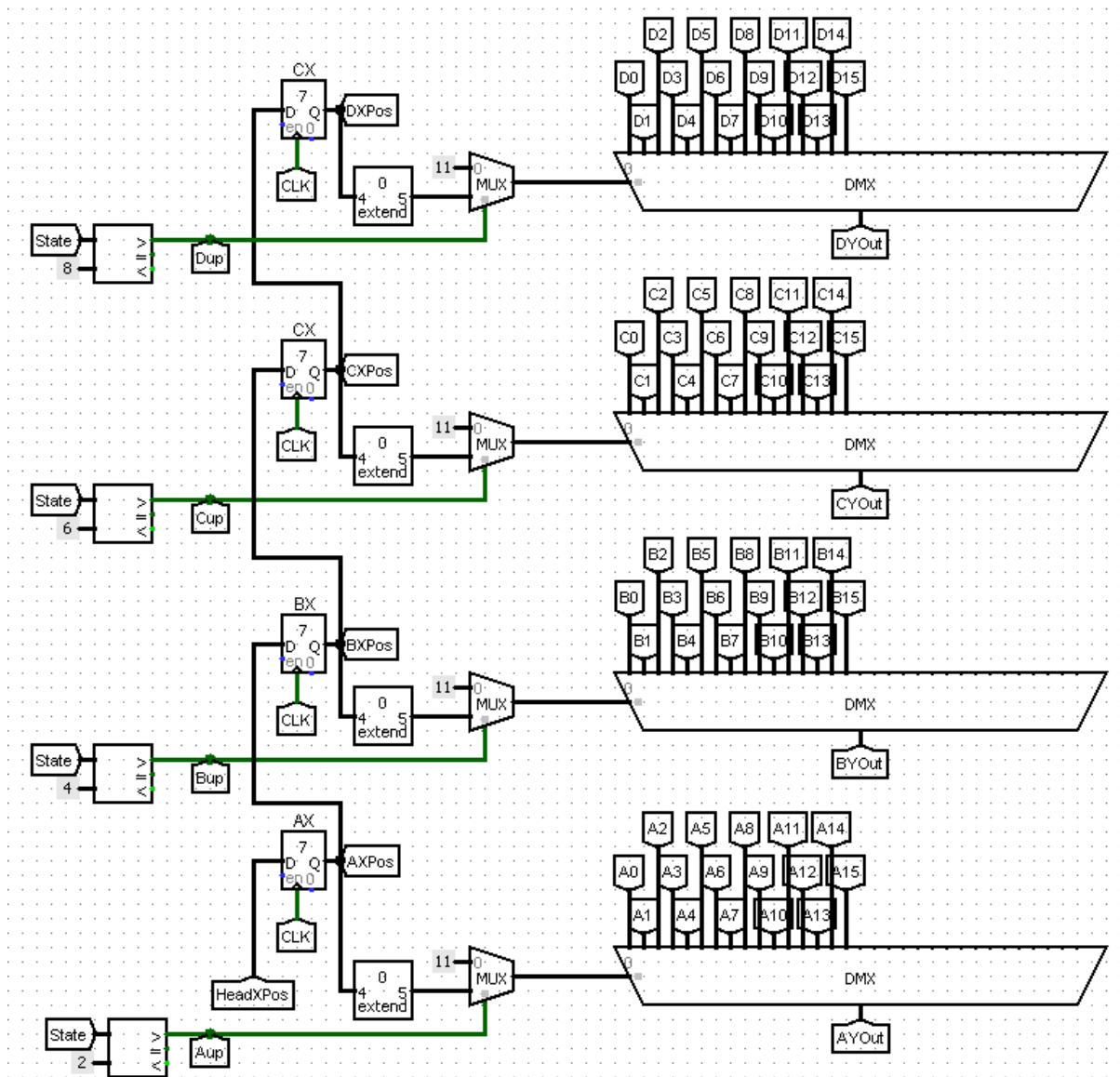
Head Controller:

This module changes the position of the head in correlation of the direction change and state values using **adder** and **subtractor**. State are different situations the snake can be in where 1 is just the body and 10 is the win state.



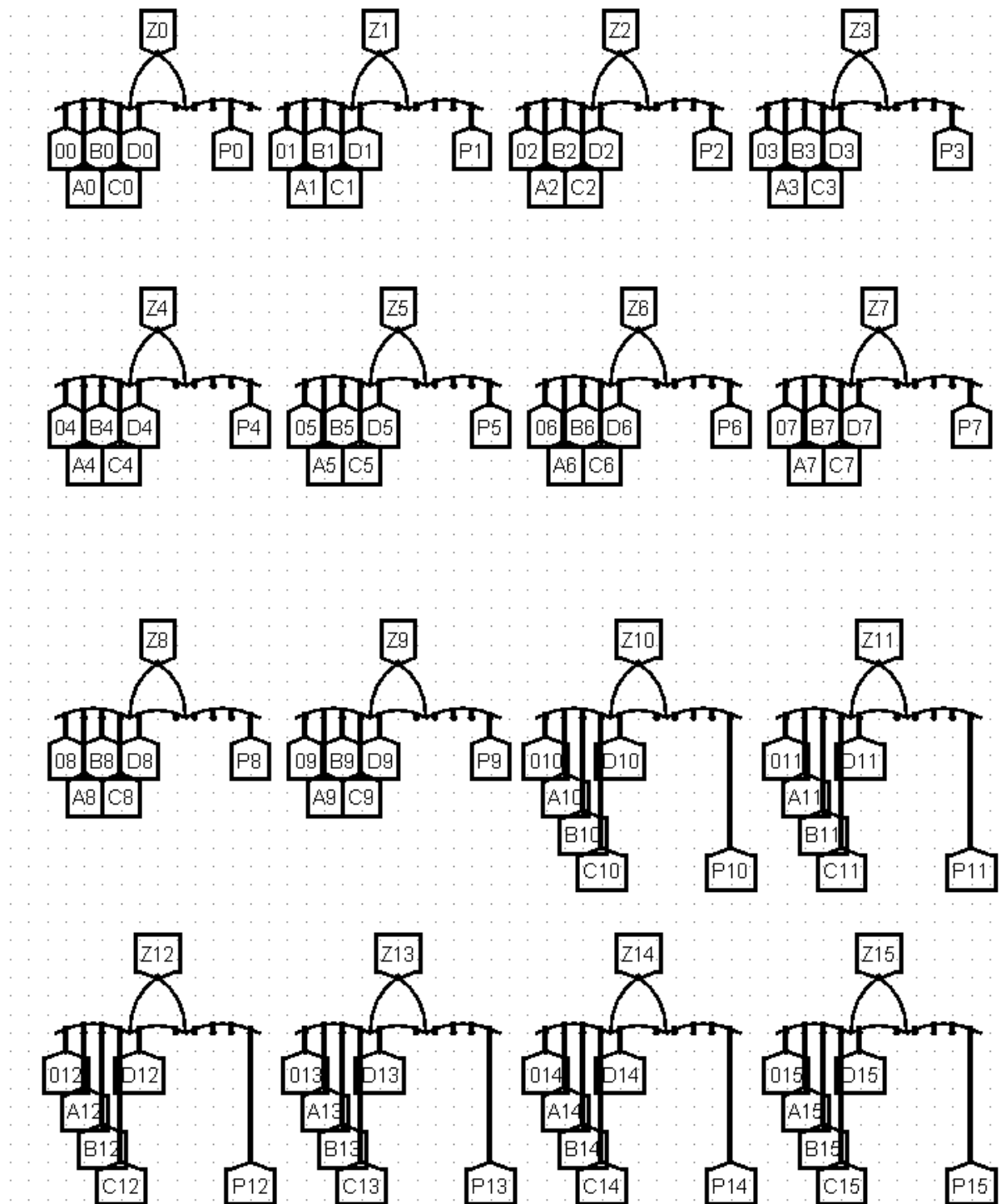
This module sends the Y output on a specific tunnel using the X position of the head.

Snake Body Y Position Info Module:

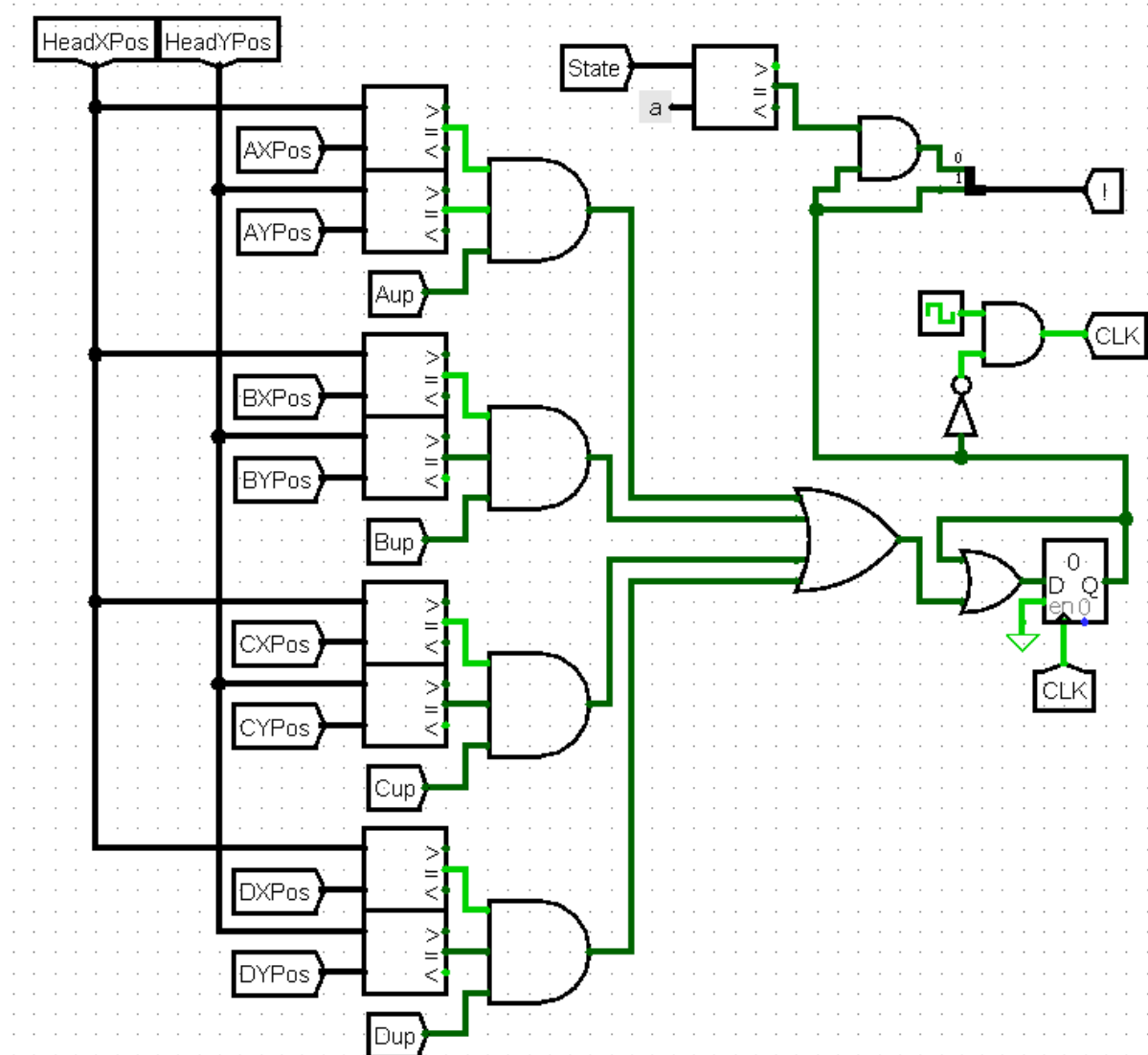


Snake Body X Position Info Module:

This module checks the state of the snake using **comparators**, then updates the body pixel positions using the head position and its previous parts position. For example, the position of the B part would be determined by the position of the A part and head. Using the positions, **demultiplexers** are used to send the value on a specific column tunnel.



This series of **OR gates** are used to send the position of the head, snake body and apple to the **Z(1-15)** terminals which then are used to update the positions on the **!(0-15)** column terminals that would update the *LED matrix*.



Collision Detector:

This module checks if when the A, B, C, D (snake body parts) are activated, the XY positions of the head of the snake matches the position of any body part using **comparators**. If it did, then it would update the **register** and change the **select bit(!)** of the visual info updater. Thus, it will show a *sad face* on the LED matrix indicating that the player has *lost*. If the snake eats 5 apples, the state value would be equal to 10 (a) and it would also change the value of the select bit. Thus, showing a *smiley face* on the LED matrix indicating player *victory*.